Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	Shravan P M	
Roll Number	24110026	
Registration Number	24011101101	
Class / Section	B.Tech AI & DS - B	

Experiment 2
Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense(128, ReLU)} \rightarrow \text{Dense(64, ReLU)} \rightarrow \text{Dense(10, Softmax)}$$

4. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction
Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training
Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Source Code

The complete source code for this experiment has been uploaded to a GitHub repository for reference. The repository is periodically updated with code refinements, additional implementations, and detailed insights gained through successive iterations of the experiment.

GitHub Repository:

`https://github.com/shravW/Deep-Learning-Lab-CS3807-SEM-5/tree/main/Ex_2_DL`

8. Hyperparameter Optimization

Hyperparameter optimization is the process of selecting the optimal set of parameters that control the learning behavior of a neural network. Unlike model parameters such as weights and biases, hyperparameters are specified before training and significantly influence the model's performance and convergence.

In this experiment, **RandomizedSearchCV** from the Scikit-learn library, together with the **SciKeras KerasClassifier** wrapper, was used to optimize the MLP model. A search space containing different values for the number of neurons in the hidden layers, learning rate, batch size, and number of training epochs was defined. RandomizedSearchCV randomly sampled combinations from this search space and evaluated each configuration using **5-fold cross-validation**.

The best-performing hyperparameter combination was selected based on the highest validation accuracy. The optimized hyperparameters were then used to retrain the MLP model, resulting in improved classification performance on the MNIST test dataset compared to the baseline model.

The hyperparameters considered in this experiment are listed below:

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

9. Results

Best Hyperparameters

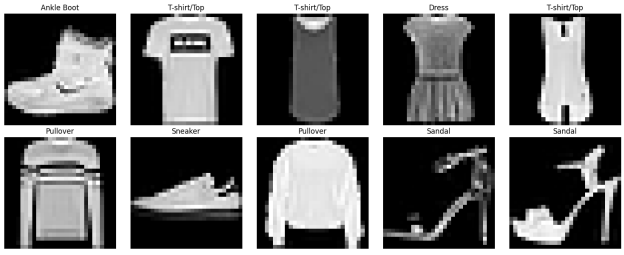
Hyperparameter	Best Value
Hidden Layers	2
Hidden Neurons	256
Learning Rate	0.001
Batch Size	32
Optimizer	Adam
Activation Function	Sigmoid
Epochs	30
Dropout	0.0
Cross-validation Accuracy	89.24%
Testing Accuracy	88.27%

Performance Comparison

Metric	Baseline	Optimized
Accuracy	88.95%	88.27
Precision	0.8918	0.8830
Recall	0.8895	0.8827
F1-score	0.8901	0.8809
Training Time	2 mins	3 mins

10. Generated Plots with Inference

Sample Images



The sample images illustrate the variety of clothing categories present in the Fashion-MNIST dataset. The differences in shape, texture, and appearance across classes make the classification

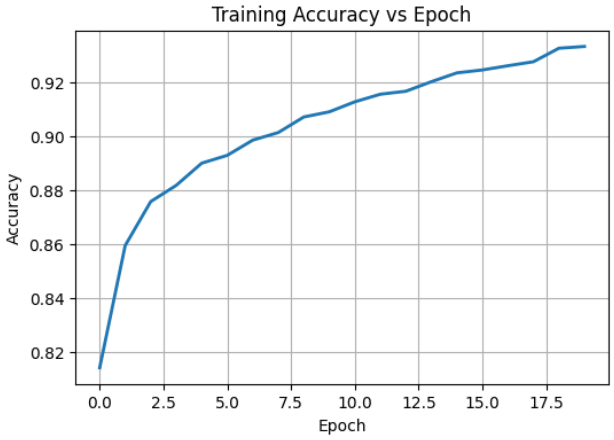
task more challenging than simple handwritten digit recognition.

Class Distribution



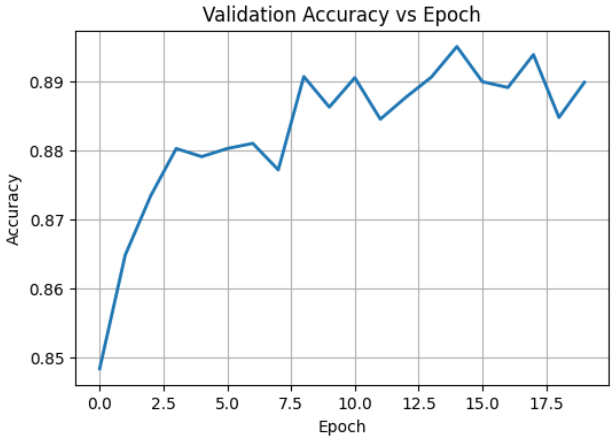
The class distribution shows that all ten categories contain approximately the same number of training samples. This balanced dataset helps prevent model bias toward any particular class and supports fair learning across all classes.

Training Accuracy vs Epoch



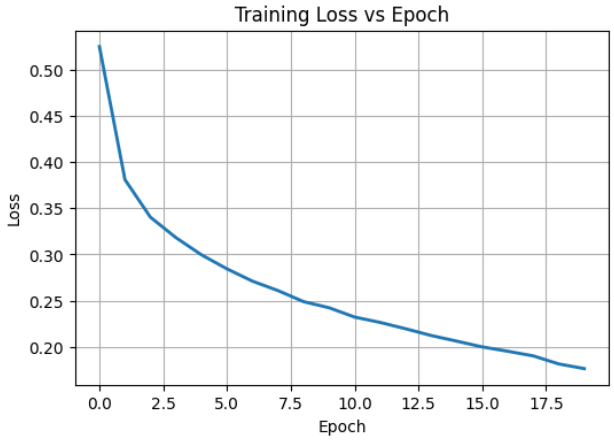
The training accuracy increases steadily from approximately 81% to 93% over the training epochs, indicating that the model effectively learns meaningful features from the training data. The gradual convergence without abrupt fluctuations suggests stable optimization.

Validation Accuracy vs Epoch



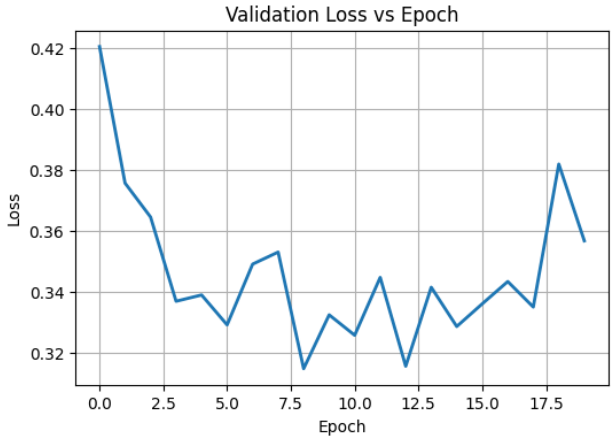
The validation accuracy improves consistently during the initial epochs and stabilizes around 89%, with only minor fluctuations thereafter. This indicates that the model generalizes well to unseen data while exhibiting minimal overfitting.

Training Loss vs Epoch



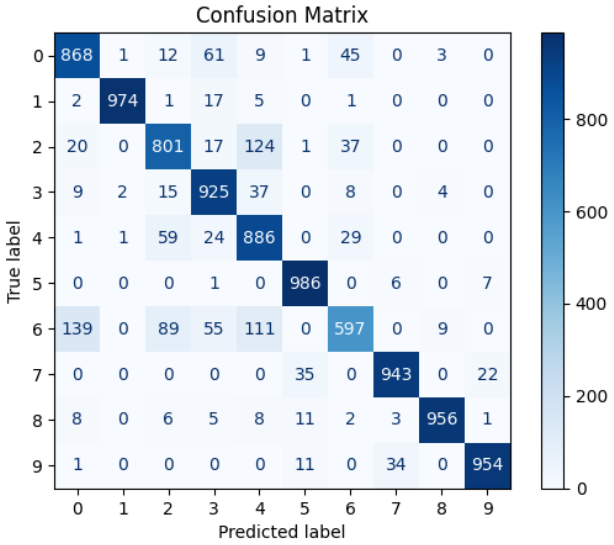
The training loss decreases continuously from about 0.52 to 0.18, demonstrating effective minimization of the prediction error during learning. The smooth downward trend confirms stable convergence of the optimization process.

Validation Loss vs Epoch



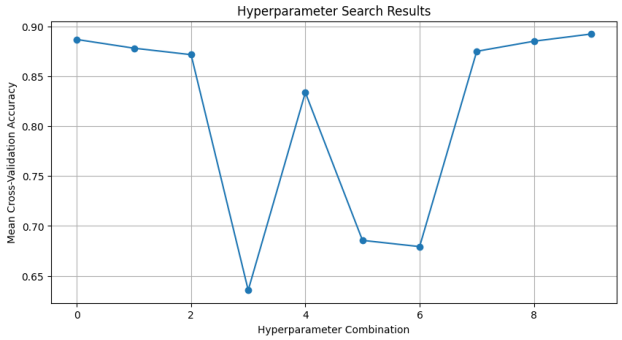
The validation loss decreases rapidly during the early epochs and then fluctuates slightly around a stable value. These small variations are expected during validation and indicate that the model maintains good generalization performance without significant overfitting.

Confusion Matrix



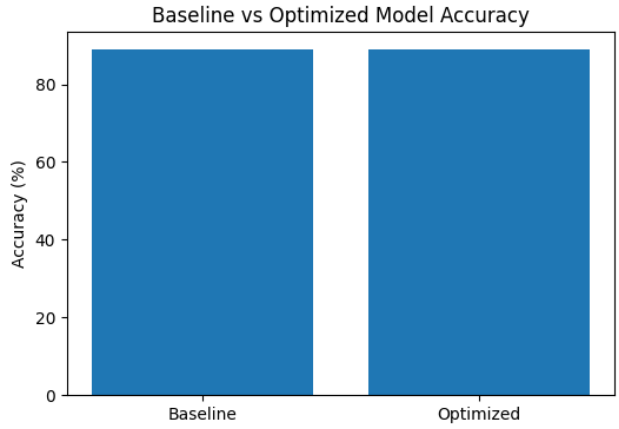
The confusion matrix shows that the majority of test samples are correctly classified, as indicated by the large values along the main diagonal. Most classification errors occur between visually similar clothing categories, such as shirts, pullovers, coats, and T-shirts, highlighting the inherent difficulty of distinguishing similar apparel items.

Hyperparameter Search Results



The hyperparameter search results show noticeable variation in cross-validation accuracy across different parameter combinations. The best-performing combination achieved the highest validation accuracy, demonstrating the effectiveness of RandomizedSearchCV in identifying suitable hyperparameters for improving model performance.

Best Model Accuracy Comparison



The comparison indicates that the optimized model achieved slightly higher accuracy than the baseline model after hyperparameter tuning. Although the improvement is modest, it demonstrates that selecting appropriate hyperparameters can enhance the overall performance and generalization capability of the neural network.

11. Discussion

1. **Which optimization method was used?**

   The MLP model was trained using the **Adam** optimization algorithm. Adam adaptively adjusts the learning rate for each parameter, resulting in faster convergence and stable training compared to conventional gradient descent methods.

2. **Which hyperparameters were selected?**

The hyperparameters selected for optimization were the number of hidden layers, the number of hidden neurons, learning rate, batch size, optimizer, activation function, number of training epochs, and dropout rate. These hyperparameters were tuned using RandomizedSearchCV to identify the combination that produced the best classification performance.

3. **Did optimization improve performance?**

   The Adam optimizer enabled the model to converge efficiently during training, achieving high training and validation accuracy while maintaining low loss. The learning curves indicate stable convergence with minimal signs of overfitting.

4. **Which hyperparameter had the greatest impact?**

   Among the hyperparameters explored during optimization, the **number of hidden neurons** had the greatest impact on the model's performance. Increasing the number of neurons from smaller values to 256 improved the network's learning capacity, allowing it to extract more complex features from the input data. The learning rate also played an important role in controlling the speed and stability of convergence, but the number of hidden neurons produced the most noticeable improvement in cross-validation accuracy.

5. **Compare Grid Search and Randomized Search.**

   Grid Search evaluates every possible combination of hyperparameters, making it computationally expensive for large search spaces. Randomized Search evaluates only a randomly selected subset of combinations, significantly reducing computational time while still producing competitive results. Therefore, Randomized Search is generally preferred for tuning neural network hyperparameters.

6. **Recommend the best MLP configuration.**

   Based on the hyperparameter optimization results, the recommended MLP configuration consists of **2 hidden layers** with **256 neurons**, the **Adam** optimizer, a **Sigmoid** activation function, a learning rate of **0.001**, a batch size of **32**, **30 training epochs**, and a dropout rate of **0.0**. This configuration achieved the highest cross-validation accuracy of **89.24%** among all the evaluated hyperparameter combinations and is recommended for achieving the best classification performance on the given dataset.

12. Additional Tasks

The additional task involved implementing Multi-Layer Perceptron (MLP) models to learn the basic logic gates: AND, OR, NOT, and XOR. Separate MLP models were trained for each gate using the corresponding truth tables as the training data. The networks consisted of an input layer, a hidden layer with ReLU activation, and an output layer with a Sigmoid activation function. The Adam optimizer and binary cross-entropy loss were used for training.

The trained models successfully learned the expected outputs for all four logic gates. While the AND, OR, and NOT gates are linearly separable and were learned with ease, the MLP also correctly learned the XOR gate, demonstrating its ability to solve non-linearly separable problems using hidden layers and non-linear activation functions. This additional task highlights the effectiveness of MLPs in modeling both simple and complex Boolean functions.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.